

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250403>

Outline : Reinforcement learning {TF support} (code [samples](#))

- Definitions / Motivation
- Policy Search
- Introduction to OpenAI Gym
- Neural Network Policies
- Evaluating Actions: The Credit Assignment Problem
- Policy Gradients
- Takeaways

—

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 ([Available online](#)) / **Chapter 18**

Definition / Motivation (1)

Playing Atari with Deep Reinforcement Learning (Videos)

Human-level control through deep reinforcement learning

In Reinforcement Learning, a software *agent* makes observations and takes *actions* within an *environment*, and in return it receives *rewards*. Its objective is to learn to act in a way that will maximize its expected rewards over time.

Definition / Motivation (2)

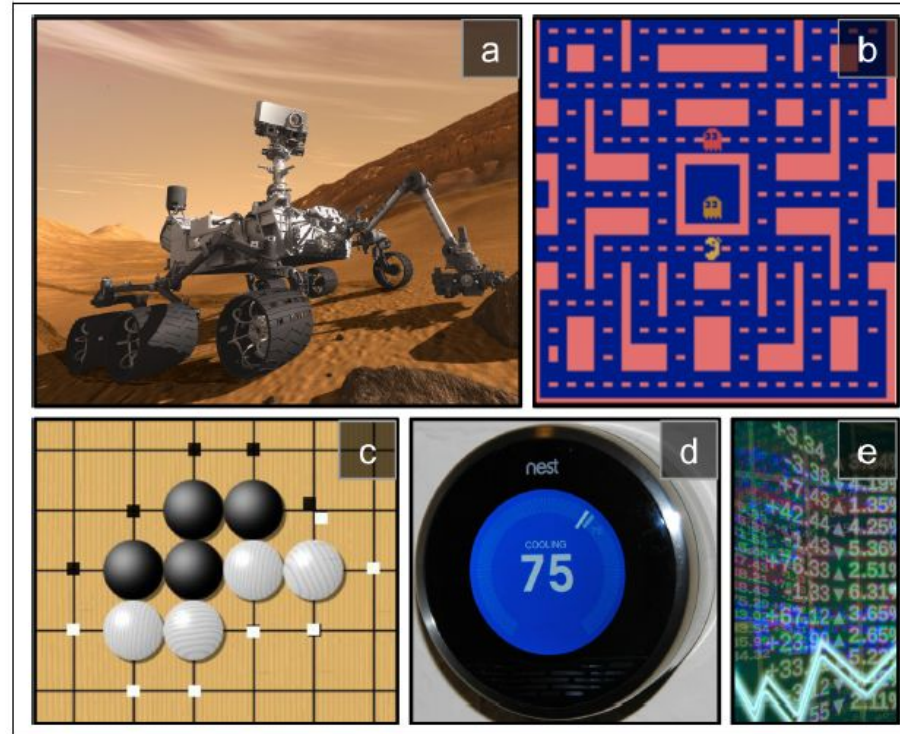


Figure 18-1. Reinforcement Learning examples: (a) robotics, (b) Ms. Pac-Man, (c) Go player, (d) thermostat, (e) automatic trader⁵

Policy Search (1)

The algorithm a software agent uses to determine its actions is called its *policy*

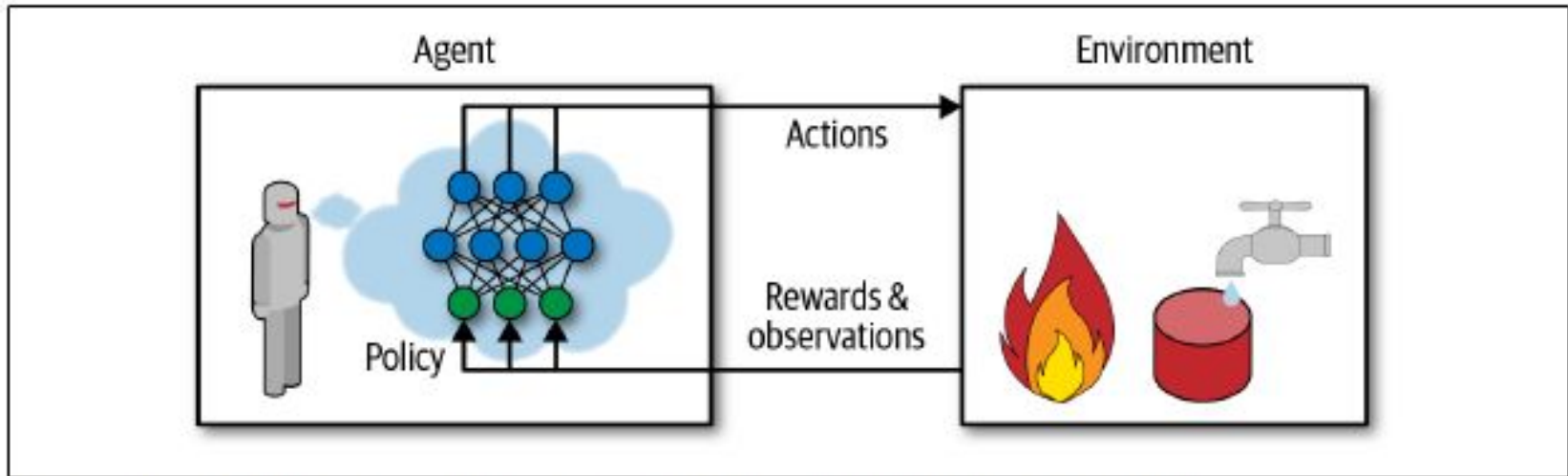


Figure 18-2. Reinforcement Learning using a neural network policy

Policy Search (2)

The policy can be any algorithm, and it does not have to be deterministic. In some cases it does not even have to observe the environment.

Agent: robotic vacuum cleaner.

Reward: amount of dust it picks up in 30 minutes.

Policy: Move forward with some probability p every second, or randomly rotate (a random angle between $-r$ and $+r$) left or right with probability $1 - p$. So, *stochastic policy*.

The robot will have an erratic trajectory, which guarantees that it will eventually get to any place it can reach and pick up all the dust.

The question is, how much dust will it pick up in 30 minutes?

Policy Search (3)

Training: Two policy parameters: the probability p and the angle range r . One possible learning algorithm to try out many different values for these parameters, and pick the combination that performs best.

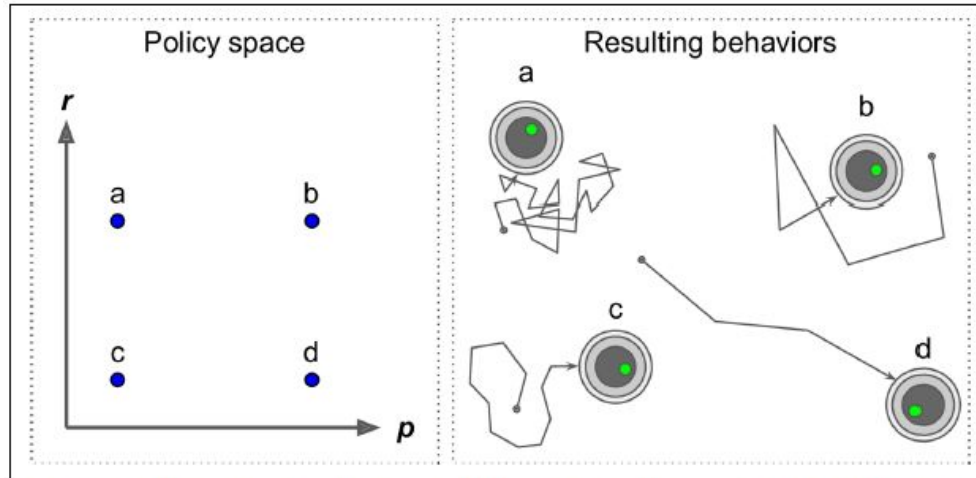


Figure 18-3. Four points in policy space (left) and the agent's corresponding behavior (right)

Introduction to OpenAI Gym: Code samples

Gymnasium

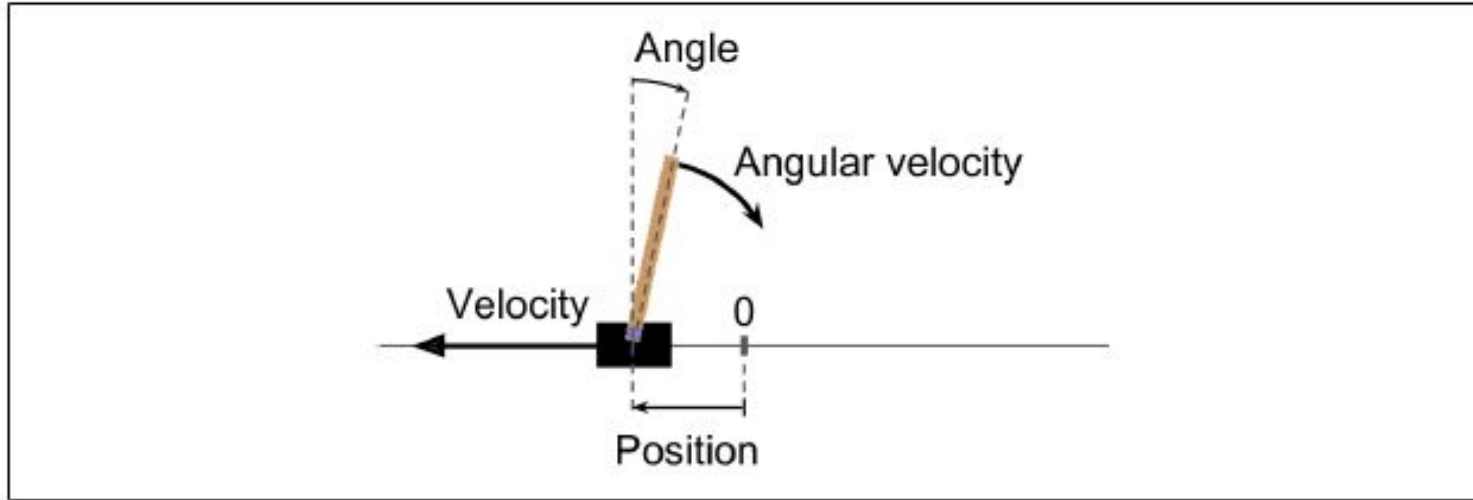


Figure 18-4. The CartPole environment

Basic policy

Simple hardcoded policy that accelerates left when the pole is leaning toward the left and accelerates right when the pole is leaning toward the right. To run this policy to see the average rewards it gets over 500 episodes:

```
def basic_policy(obs):
    angle = obs[2]
    return 0 if angle < 0 else 1
```

```
totals = []
for episode in range(500):
    episode_rewards = 0
    obs = env.reset()
    for step in range(200):
        action = basic_policy(obs)
        obs, reward, done, info = env.step(action)
        episode_rewards += reward
        if done:
            break
    totals.append(episode_rewards)
```

```
>>> import numpy as np
>>> np.mean(totals), np.std(totals), np.min(totals), np.max(totals)
(41.718, 8.858356280936096, 24.0, 68.0)
```

Neural Network Policies (1)

Neural network policy - takes an observation as input, and it will output the action to be executed.

Estimates a probability for each action, and then we will **select an action randomly**, according to the estimated probabilities.

In the case of the CartPole environment, there are just two possible actions (left or right), so only one output neuron. It will output the probability p of action 0 (left), and of course the probability of action 1 (right) will be $1 - p$. For example, if it outputs 0.7, then we will pick action 0 with 70% probability, or action 1 with 30% probability.

Neural Network Policies (2)

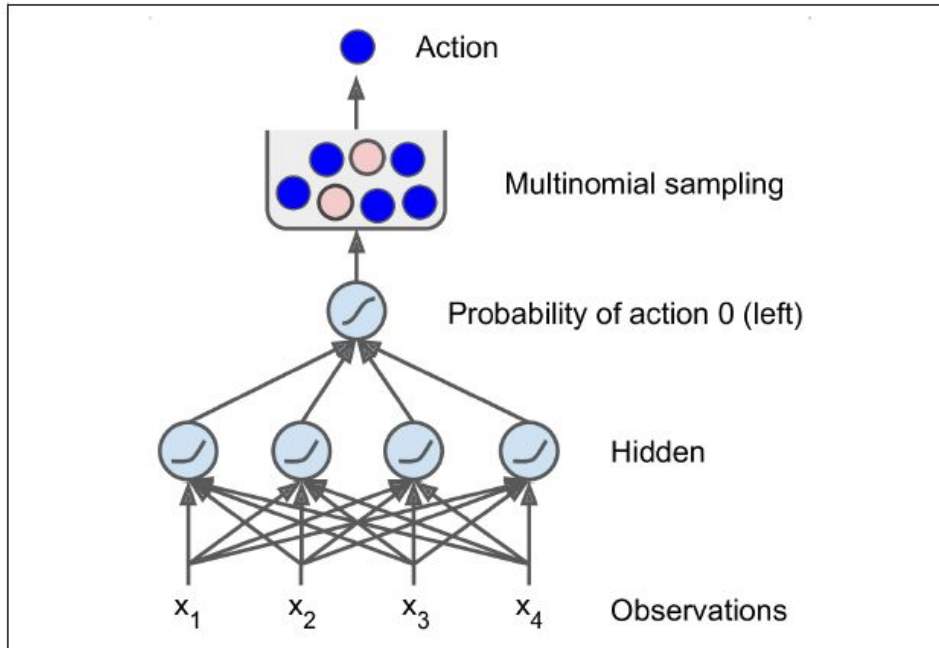


Figure 18-5. Neural network policy

Neural Network Policies (3)

Why **to pick a random action based on the probabilities** given by the neural network, rather than just picking the action with the highest score? This approach lets the agent find the right balance between *exploring* new actions and *exploiting* the actions that are known to work well.

Also note that in this particular environment, the past actions and observations can safely be ignored, since **each observation contains the environment's full state**. If there were **some hidden state**, then you might need to **consider past actions and observations as well**.

Neural Network Policies (4)

```
import tensorflow as tf
from tensorflow import keras

n_inputs = 4 # == env.observation_space.shape[0]

model = keras.models.Sequential([
    keras.layers.Dense(5, activation="elu", input_shape=[n_inputs]),
    keras.layers.Dense(1, activation="sigmoid"),
])
```

How to train it?

Evaluating Actions: The Credit Assignment Problem (1)

- If we knew what the best action was at each step, we could train the neural network as usual, by minimizing the cross entropy between the estimated probability distribution and the target probability distribution. It would just be regular supervised learning.
- In Reinforcement Learning the only guidance the agent gets is through rewards, and rewards are typically sparse and delayed.
- If the agent manages to balance the pole for 100 steps, how can it know which of the 100 actions it took were good, and which of them were bad? All it knows is that the pole fell after the last action, but surely this last action is not entirely responsible.
- **Credit assignment problem:** when the agent gets a reward, it is hard for it to know which actions should get credited (or blamed) for it.

Evaluating Actions: The Credit Assignment Problem (2)

- A common strategy: to **evaluate an action based on the sum of all the rewards that come after it**, usually applying a discount factor γ (gamma) at each step.
- This sum of discounted rewards is called the action's return.

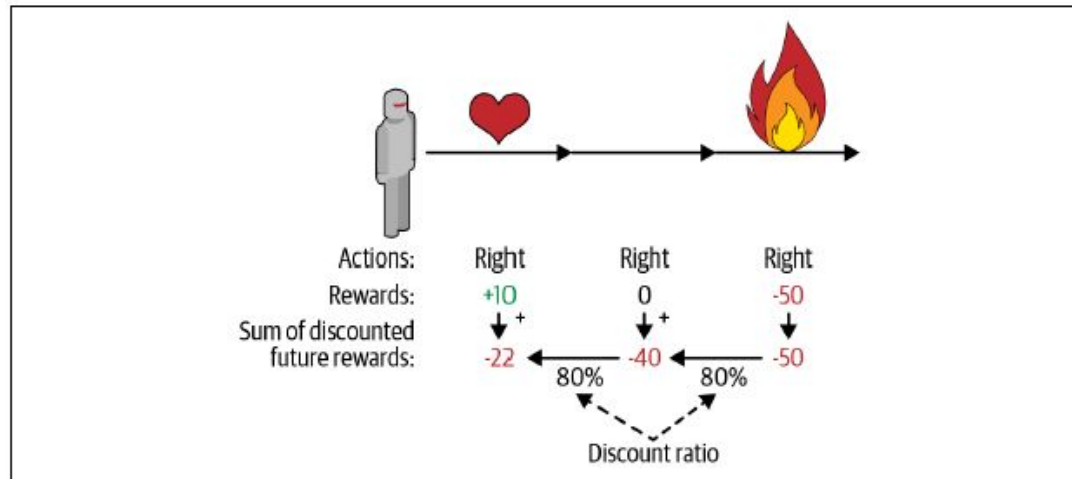


Figure 18-6. Computing an action's return: the sum of discounted future rewards

Evaluating Actions: The Credit Assignment Problem (3)

- If an agent decides to go right three times in a row and gets +10 reward after the first step, 0 after the second step, and finally -50 after the third step, then assuming we use a discount factor $\gamma = 0.8$, the **first action** will have a **return** of $10 + \gamma \times 0 + \gamma^2 \times (-50) = -22$.

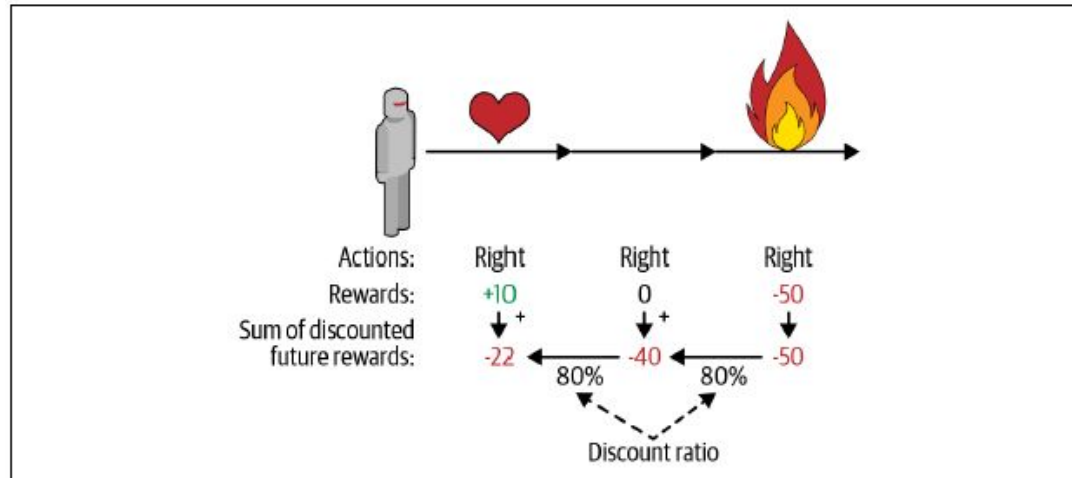


Figure 18-6. Computing an action's return: the sum of discounted future rewards

Evaluating Actions: The Credit Assignment Problem (4)

- If the discount factor is close to 0, then future rewards will not count for much compared to immediate rewards. Conversely, if the discount factor is close to 1, then rewards far into the future will count almost as much as immediate rewards. Typical discount factors vary from 0.9 to 0.99.
- With a discount factor of 0.95, rewards 13 steps into the future count roughly for half as much as immediate rewards (since $0.95^{13} \approx 0.5$), while with a discount factor of 0.99, rewards 69 steps into the future count for half as much as immediate rewards.
- In the CartPole environment, actions have fairly short-term effects, so choosing a discount factor of 0.95 seems reasonable.

Evaluating Actions: The Credit Assignment Problem (5)

- A good action may be followed by several bad actions that cause the pole to fall quickly, resulting in the good action getting a low return (similarly, a good actor may sometimes star in a terrible movie).
- However, if we play the game enough times, on average good actions will get a higher return than bad ones.
- We want to estimate how much better or worse an action is, compared to the other possible actions, on average. This is called the **action advantage**.
 - For this, we must run many episodes and normalize all the action returns (by subtracting the mean and dividing by the standard deviation). After that, we can reasonably assume that actions with a negative advantage were bad while actions with a positive advantage were good.

Policy Gradients (PG) (1)

- PG algorithms optimize the parameters of a policy by following the gradients toward higher rewards.
- REINFORCE algorithms (1992)
 - First, let the neural network policy play the game several times, and at each step, compute the gradients that would make the chosen action even more likely—but don't apply these gradients yet.
 - Once you have run several episodes, compute each action's advantage (using the method described in the previous section).
 - If an action's advantage is positive, it means that the action was probably good, and you want to apply the gradients computed earlier to make the action even more likely to be chosen in the future. However, if the action's advantage is negative, it means the action was probably bad, and you want to apply the opposite gradients to make this action slightly less likely in the future. The solution is simply to multiply each gradient vector by the corresponding action's advantage.
 - Finally, compute the mean of all the resulting gradient vectors, and use it to perform a Gradient Descent step.

Policy Gradients (2)

```
def play_one_step(env, obs, model, loss_fn):
    with tf.GradientTape() as tape:
        left_proba = model(obs[np.newaxis])
        action = (tf.random.uniform([1, 1]) > left_proba)
        y_target = tf.constant([[1.]]) - tf.cast(action, tf.float32)
        loss = tf.reduce_mean(loss_fn(y_target, left_proba))
    grads = tape.gradient(loss, model.trainable_variables)
    obs, reward, done, info = env.step(int(action[0, 0].numpy()))
    return obs, reward, done, grads
```

```
def play_multiple_episodes(env, n_episodes, n_max_steps, model, loss_fn):
    all_rewards = []
    all_grads = []
    for episode in range(n_episodes):
        current_rewards = []
        current_grads = []
        obs = env.reset()
        for step in range(n_max_steps):
            obs, reward, done, grads = play_one_step(env, obs, model, loss_fn)
            current_rewards.append(reward)
            current_grads.append(grads)
            if done:
                break
        all_rewards.append(current_rewards)
        all_grads.append(current_grads)
    return all_rewards, all_grads
```

Code returns a list of reward lists (one reward list per episode, containing one reward per step), as well as a list of gradient lists (one gradient list per episode, each containing one tuple of gradients per step and each tuple containing one gradient tensor per trainable variable).

Policy Gradients (3)

The algorithm will use the `play_multiple_episodes()` function to play the game several times (e.g., 10 times), then it will go back and look at all the rewards, discount them, and normalize them.

```
def discount_rewards(rewards, discount_factor):  
    discounted = np.array(rewards)  
    for step in range(len(rewards) - 2, -1, -1):  
        discounted[step] += discounted[step + 1] * discount_factor  
    return discounted  
  
def discount_and_normalize_rewards(all_rewards, discount_factor):  
    all_discounted_rewards = [discount_rewards(rewards, discount_factor)  
                             for rewards in all_rewards]  
    flat_rewards = np.concatenate(all_discounted_rewards)  
    reward_mean = flat_rewards.mean()  
    reward_std = flat_rewards.std()  
    return [(discounted_rewards - reward_mean) / reward_std  
            for discounted_rewards in all_discounted_rewards]
```

Policy Gradients (4)

Hyperparameters: running 150 training iterations, playing 10 episodes per iteration, and each episode will last at most 200 steps. We will use a discount factor of 0.95:

```
n_iterations = 150  
n_episodes_per_update = 10  
n_max_steps = 200  
discount_factor = 0.95
```

Also need an optimizer and the loss function. A regular Adam optimizer with learning rate 0.01 will do just fine, and we will use the binary cross-entropy loss function because we are training a binary classifier (there are two possible actions: left or right):

```
optimizer = keras.optimizers.Adam(lr=0.01)  
loss_fn = keras.losses.binary_crossentropy
```

Policy Gradients (5)

```
for iteration in range(n_iterations):
    all_rewards, all_grads = play_multiple_episodes(
        env, n_episodes_per_update, n_max_steps, model, loss_fn)
    all_final_rewards = discount_and_normalize_rewards(all_rewards,
                                                       discount_factor)

    all_mean_grads = []
    for var_index in range(len(model.trainable_variables)):
        mean_grads = tf.reduce_mean(
            [final_reward * all_grads[episode_index][step][var_index]
             for episode_index, final_rewards in enumerate(all_final_rewards)
             for step, final_reward in enumerate(final_rewards)], axis=0)
        all_mean_grads.append(mean_grads)
    optimizer.apply_gradients(zip(all_mean_grads, model.trainable_variables))
```

Policy Gradients (6)

```
for iteration in range(n_iterations):
    all_rewards, all_grads = play_multiple_episodes(
        env, n_episodes_per_update, n_max_steps, model, loss_fn)
    all_final_rewards = discount_and_normalize_rewards(all_rewards,
                                                       discount_factor)

    all_mean_grads = []
    for var_index in range(len(model.trainable_variables)):
        mean_grads = tf.reduce_mean(
            [final_reward * all_grads[episode_index][step][var_index]
             for episode_index, final_rewards in enumerate(all_final_rewards)
             for step, final_reward in enumerate(final_rewards)], axis=0)
        all_mean_grads.append(mean_grads)
    optimizer.apply_gradients(zip(all_mean_grads, model.trainable_variables))
```

- At each training iteration, this loop calls the `play_multiple_episodes()` function, which plays the game 10 times and returns all the rewards and gradients for every episode and step.
- Then we call the `discount_and_normalize_rewards()` to compute each action's normalized advantage (which in the code we call the `final_reward`). This provides a measure of how good or bad each action actually was, in hindsight.
- Next, we go through each trainable variable, and for each of them we compute the weighted mean of the gradients for that variable over all episodes and all steps, weighted by the `final_reward`.
- Finally, we apply these mean gradients using the optimizer: the model's trainable variables will be tweaked, and hopefully the policy will be a bit better.

Takeaways

- Hardcoded, stochastic, NN-based policies.
- Action evaluation, selecting discount factor.
- REINFORCE algorithms.